

Second Research/Programming Assignment

Dixin Zhang, 2026/4/20

Question1

Self-Play vs. Fixed Opponent Training in Tic-Tac-Toe



Key Insight 1:

Self-Play Converges to Equilibrium

- Both agents **improve** simultaneously
- harder to exploit each other
- **draw** becomes the dominant outcome



Key Insight 2:

The Strategy Learned is Robust but Conservative

- **Win** if possible
- **draw** if not
- avoid losing moves at all costs
- classic **minimax** behavior



Key Insight 3:

Trade-off — Robustness vs. Exploitation

- Self-play model = near-optimal, hard to beat
- **Fixed-opponent** model = specialized, better at exploiting specific weaknesses
- Self-play agent may **underperform** against a weak opponent it was never trained to exploit



Takeaway → Self-play learns the structure of the game; **fixed-opponent training** learns to beat that opponent

Question2: Exploiting Symmetry in Tic-Tac-Toe Learning



Key Insight 1: Symmetry Reduces the State Space

- Rotations & reflections collapse equivalent boards into one state
- Fewer states to learn → faster convergence → better generalization
- Can be implemented via preprocessing or NN architecture



Key Insight 2: If Symmetries Are Known A Priori → More Efficient Updates

- A transition from state S can simultaneously update S and its symmetric equivalent
- Doubles the learning signal without extra interactions



Key Insight 3: But — Only Valid If the Opponent Also Respects Symmetry

- If opponent has asymmetric tendencies (e.g., always prefers a corner), symmetric states have different real-world values
- Forcing symmetry = losing the ability to exploit opponent's bias



Takeaway → Symmetric states should share values only if the environment (opponent policy) is also symmetric;
→ Symmetry is a powerful shortcut — but context-dependent, not unconditional

Question3: MDP in the Real World – Examples & Intuition

Core Intuition: → MDP = repeated decision-making where actions converge to true values over many attempts
→ More experience in a state → more accurate value estimate for each action

Examples (States → Actions → Rewards)

	Example	State	Action	Reward
	Coffee	Tired	Brew espresso	Caffeine boost
	Valorant (Sova)	3v3, cooldown status, last known enemy pos.	Fire Recon Bolt / hold angle / rotate	+5 info, +100 round win, -20 wasted ability
	Dengue Response	Climate data, mosquito density, case distribution	Deploy fumigation / public campaign / hold budget	+reward for low infection rate, -penalty for outbreaks & resource cost



Key Insight: The Dengue Example Pushes the Boundary

- Agent is not a physical entity – it's a macro-level public health dispatch system
- Environment = entire urban ecosystem (climate + population + disease dynamics)
- Reward structure forces a trade-off: outbreak prevention vs. budget conservation
- Demonstrates MDP applies beyond games – to real-world policy and resource allocation

Question4: The Agent-Environment Boundary

A Design Choice, Not a Physical Law

1. The Three Levels of Abstraction

- **Low (Physical)**: Signals & Mechanics (e.g., Cell Response / Motor Voltage)
- **Mid (Tactical)**: Control & Actions (e.g., Treatment Plan / Steering Angle)
- **High (Strategic)**: Policy & Planning (e.g., Health Management / Navigation)

2. The Golden Rule

- **Strip Complexity**: Push irrelevant details into the environment.
- **Retain Control**: Keep only what's needed to solve the problem.

3. Bottom Line

"Choose the boundary that makes learning tractable."

Question5: MABS for Ads CTR Optimization

Core Content:

- **Business Scenario:** 10 different ads (Ad 1 to Ad 10) are randomly displayed to users. The goal is to identify the ad with the highest Click-Through Rate (CTR) to maximize revenue.
- **Data Source:** Kaggle Ads CTR Optimisation Dataset (10,000 rounds).
- **The Challenge:** The Exploration vs. Exploitation dilemma — should we continue testing unknown ads, or focus on exploiting the current best-performing ad?



Multi-armed Bandit / Slot Machine



► Baseline Ground Truth: The actual optimal ad is Ad 5 (CTR = 26.95%).

ϵ -Greedy Algorithm — Quick to Exploit

How It Works

- With probability $\epsilon = 0.1$, pick a random ad to explore.
- With probability 0.9, exploit the current best-performing ad.
- Simple, fast, and effective for environments where one option is clearly dominant.

68.62%

Optimal Ad Selection Rate
(6,862 / 10,000 rounds)

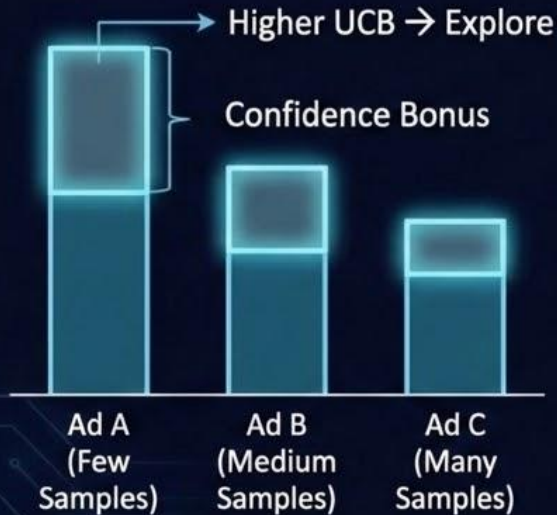
Results

- The algorithm successfully converged on Ad 5 as the optimal choice.
- Across 10,000 rounds, it selected the best ad 6,862 times — an optimal selection rate of 68.62%.



UCB Algorithm — Rigorous Exploration

How It Works



UCB assigns a confidence bonus based on how rarely an ad has been tested. The less data, the higher its upper confidence bound — pushing exploration of uncertain options.

Key Metric

42.92%

Optimal Ad Selection Rate
(4,292 / 10,000 rounds)

Results

- Identified Ad 5 as the optimal ad.
- Across 10,000 rounds, selected the optimal ad 4,292 times.

Interpretation

- Lower rate compared to ϵ -greedy reflects deliberate investment in exploration.
- More rounds spent stress-testing other ads before committing to a winner.

Comparison and Conclusion

	ϵ -Greedy	UCB
Best ad identified	Ad 5	Ad 5
True CTR of Ad 5	0.2695	0.2695
Estimated CTR of Ad 5	0.2683	0.2719
Times Ad 5 selected	6,862	4,292
Optimal-arm selection rate	68.62%	42.92%

Conclusions

- Both algorithms correctly identified Ad 5 as the optimal banner.
- ϵ -Greedy exploited the best ad faster, achieving a higher selection rate.
- UCB explored more conservatively, distributing impressions more evenly across all 10 ads.
- When one arm has a clearly dominant CTR, faster exploitation yields better practical performance.

RL Connection

This case is a direct empirical demonstration of the exploration-exploitation tradeoff at the core of Multi-Armed Bandit theory (Sutton & Barto, Ch. 2).

Final Takeaway

For this ad selection dataset, epsilon-greedy was the more effective strategy.

References

Harder, H. D. (2022, November 4). Just a moment... Just a moment...
<https://towardsdatascience.com/solving-multi-armed-bandit-problems-53c73940244a/>

Naereen. (n.d.). Jupyter notebooks by Naereen @ GitHub — SMPyBandits 0.9.6 documentation. Welcome to SMPyBandits documentation! — SMPyBandits 0.9.6 documentation. <https://smpybandits.github.io/notebooks/README.html>

Siddiqui, M. I. (n.d.). ADS_CTR_OPTIMISTAION. Kaggle: The World's AI Proving Ground. <https://www.kaggle.com/datasets/moimransiddiqui/ads-ctr-optimistaion>

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press.

Thomas, F. (n.d.). A Jupyter notebook taking a look at the multi-armed bandit problem. Gist. <https://gist.github.com/pstthomas/4cf8b490a74ec1d7aee083b72c40c61a>

Thank you!
